

Example 2: C++ new and delete Operator for Arrays

```
// C++ Program to store GPA of n number of students and display it
// where n is the number of students entered by the user

#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter total number of students: ";
    cin >> num;
    float* ptr;

    // memory allocation of num number of floats.
    ptr = new float[num];

    cout << "Enter GPA of students." << endl;
    for (int i = 0; i < num; ++i) {
        cout << "Student" << i + 1 << ": ";
        cin >> *(ptr + i);
    }

    cout << "\nDisplaying GPA of students." << endl;
    for (int i = 0; i < num; ++i) {
        cout << "Student" << i + 1 << " : " << *(ptr + i) << endl;
    }

    // ptr memory is released
    delete[] ptr;

    return 0;
}
```

Output

```
Enter total number of students: 4
Enter GPA of students.
Student1: 3.6
Student2: 3.1
```



```
Student3: 3.9
```

```
Student4: 2.9
```

```
Displaying GPA of students.
```

```
Student1 :3.6
```

```
Student2 :3.1
```

```
Student3 :3.9
```

```
Student4 :2.9
```

In this program, we have asked the user to enter the number of students and store it in the `num` variable.

Then, we have allocated the memory dynamically for the `float` array using `new`. We enter data into the array (and later print them) using pointer notation.

After we no longer need the array, we deallocate the array memory using the code `delete[] ptr;`.

Notice the use of `[]` after `delete`. We use the square brackets `[]` in order to denote that the memory deallocation is that of an array.

Example 3: C++ new and delete Operator for Objects

```
#include <iostream>
using namespace std;
```

```
class Student {
```

```
    int age;
```

```
public:
```

```
    // constructor initializes age to 12
```

```
    Student() : age(12) {}
```

```
    void getAge() {
```

```
        cout << "Age = " << age << endl;
```

```
    }
```

```
};
```

```
int main() {
```



```
// dynamically declare Student object
Student* ptr = new Student();

// call getAge() function
ptr->getAge();

// ptr memory is released
delete ptr;

return 0;
}
```

Output

```
Age = 12
```

In this program, we have created a `Student` class that has a private variable `age`.

We have initialized `age` to 12 in the default constructor `Student()` and print its value with the function `getAge()`.

In `main()`, we have created a `Student` object using the `new` operator and use the pointer `ptr` to point to its address.

The moment the object is created, the `Student()` constructor initializes `age` to 12.

We then call the `getAge()` function using the code:

```
ptr->getAge();
```

Notice the arrow operator `->`. This operator is used to access class members using pointers.

C++ this Pointer

In C++ programming, **this** is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.

- It can be used to **pass current object as a parameter to another method.**
- It can be used to **refer current class instance variable.**
- It can be used to **declare indexers.**

C++ this Pointer Example

Let's see the example of this keyword in C++ that refers to the fields of current class.

```
1.    #include <iostream>
2.    using namespace std;
3.    class Employee {
4.    public:
5.        int id; //data member (also instance variable)
6.        string name; //data member(also instance variable)
7.        float salary;
8.        Employee(int id, string name, float salary)
9.        {
10.           this->id = id;
11.           this->name = name;
12.           this->salary = salary;
13.        }
14.        void display()
15.        {
16.           cout<<id<<" "<<name<<" "<<salary<<endl;
17.        }
18.    };
19.    int main(void) {
20.        Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of Employee
21.        Employee e2=Employee(102, "Nakul", 59000); //creating an object of Employee
22.        e1.display();
23.        e2.display();
24.        return 0;
25.    }
```

Output:

```
101 Sonoo 890000
102 Nakul 59000
```

C++ Operators Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type. For example, C++ provides the ability to add the variables of the user-defined data type that is applied to the built-in data types.

The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

- Scope operator (::)
- Sizeof
- member selector (.)
- member pointer selector (*)
- ternary operator (?:)

Syntax of Operator Overloading

```
return_type class_name :: operator op(argument_list)
{
    // body of the function.
}
```

Where the **return type** is the type of value returned by the function.

class_name is the name of the class.

operator op is an operator function where op is the operator being overloaded, and the operator is the keyword.

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- The overloaded operator contains atleast one operand of the user-defined data type.
- We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.
- When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
- When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

C++ Operators Overloading Example

Let's see the simple example of operator overloading in C++. In this example, void operator ++ () operator function is defined (inside Test class).

// program to overload the unary operator ++.

```
#include <iostream>
using namespace std;
class Test
{
private:
    int num;
public:
    Test(): num(8){}
    void operator ++() {
        num = num+2;
    }
    void Print() {
        cout<<"The Count is: "<<num;
```